



C프로그래밍 (CSE2003)

4-1. Functions

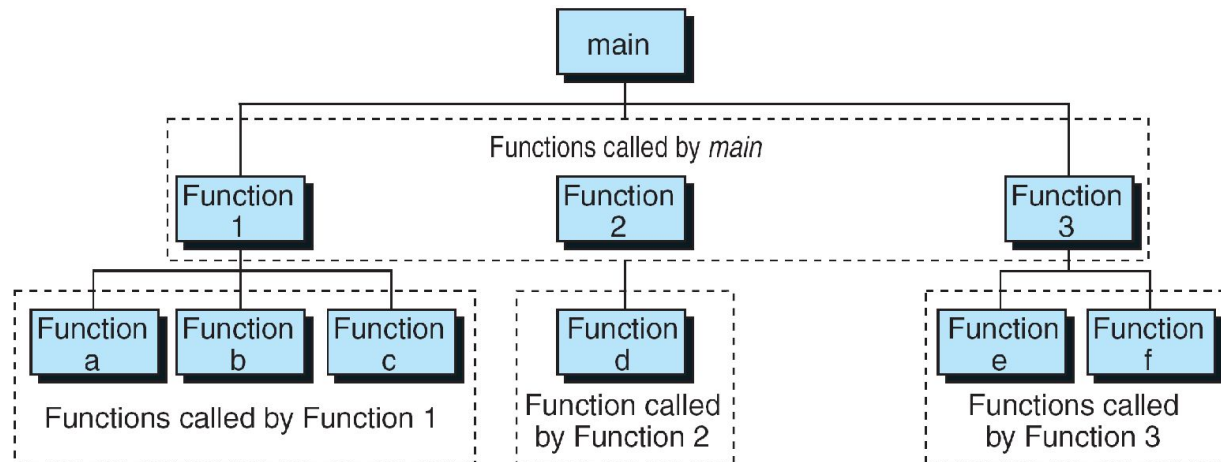
Designing Structured Programs

■ Top-down design

- 어려운 문제를 더 단순하고 쉬운 작은 문제로 나누어서 단계적으로 구체화하는 방법
- 각 단순화한 문제를 계속하여 단순한 프로그램으로 분리하여 구현함으로써 단계적으로 프로그램을 하게 함

■ Structured Programming

- 프로그램의 기능들을 세분화한 후에 순차, 선택, 반복 기법을 이용하여 프로그램 하는 방법
- 구조 차트를 그려서 기능을 세분화 함



<구조차트>

C언어의 함수

- C언어에서 함수는 특정한 작업을 수행하기 위하여 불려지는 독립적인 단위 프로그램.
- C언어는 함수를 통해 top-down design의 방식 취함
 - C 프로그램은 하나 또는 하나 이상의 함수를 통해 만들어지며, **main 함수는 반드시 하나.**
- C언어에서의 함수사용은 다음과 같은 장점 제공
 - 문제를 분리하여 **단순화** 시킴
 - 함수는 한곳 이상에서 필요로 하는 코드를 **재사용** 할 수 있는 방법
 - 이미 제공되어 있는 library를 통해 사전에 필요한 다양한 함수를 제공받아 프로그램 개발에 **편의성** 지님
 - 함수는 데이터를 보호
 - ◆ 함수 안에서 선언된 지역변수는 그 함수 안에서만 사용 가능하며, 그 함수가 실행될 때만 이용되어짐
- 함수의 종류
 - Standard functions
 - User-defined functions

우리는 이미 scanf, printf 등을 통해 함수를 사용해 보았다. 이러한 함수들은 **사용자 정의(User-defined)**에 의해 만들어진 함수가 아니라 C에서 제공하는 **표준 함수(Standard functions)**이다.

Designing Structured Programs

■ 함수

- 주 프로그램으로부터 인수(argument)를 전달 받아 일련의 작업을 수행한 후 생성된 결과를 주프로그램(main)으로 전달하는 하나의 단위 프로그램

■ 함수를 사용하는 이유

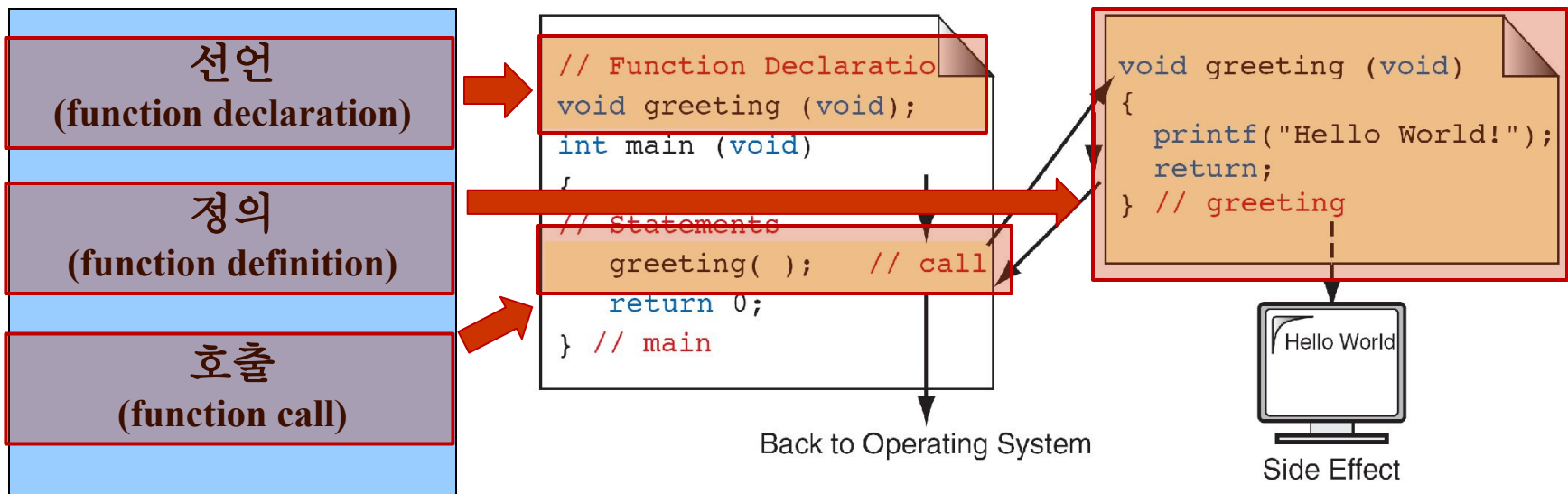
- 반복되는 코드 작성을 없애 줌
- 프로그램을 이해하기 쉽고 이식이 용이하게 함
- 모듈성(modularity) 향상시킴

■ 함수 간의 통신

- 인자의 전달과 리턴되는 값
- 외부 변수를 이용

사용자 정의 함수(User-defined Functions)

- 사용자 정의 함수는 사용자에게 의해 원하는 의도에 따라 내용을 구성하여 호출하여 사용할 수 있음
- C언어에서 함수를 사용하려면 다음과 같은 요소 필요



예제 프로그램 1 - 사용자 정의 함수

예제 소스

```
#include <stdio.h>

float calculate_Circumference(const float, float);

int main(void)
{
    float circ;
    float radius;
    const float PI = 3.1416;

    printf("\nPlease enter the value of the radius: ");
    scanf("%f", &radius);

    circ = calculate_Circumference(PI, radius);

    printf("\nRadius is :           %10.2f", radius);
    printf("\nCircumference is :      %10.2f", circ);
    printf("\n");

    return 0;
}

float calculate_Circumference(const float PI, float radius)
{
    float result;
    result = 2 * PI * radius;
    return result;
}
```

선언부
(function declaration)

호출
(function call)

정의
(function definition)

결과

```
kimjihwan@csp:~/test_dr$ gcc -o a.out circle.c
kimjihwan@csp:~/test_dr$ ./a.out

Please enter the value of the radius: 23

Radius is :                23.00
Circumference is :         144.51
Area is :                  1661.91
kimjihwan@csp:~/test_dr$
```

예제 프로그램 2 - 사용자 정의 함수 (비교)

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float circ;
```

```
    float radius;
```

```
    const float PI = 3.1416;
```

```
    printf("\nPlease enter the value of the radius: ");
```

```
    scanf("%f", &radius);
```

```
    circ = 2 * PI * radius;
```

```
    printf("\nRadius is :           %10.2f", radius);
```

```
    printf("\nCircumference is :       %10.2f", circ);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

예제 소스

main()에서 직접 계산

결과

```
kimjihwan@cspro:~/test_dr$ gcc -o b.out circle2.c
kimjihwan@cspro:~/test_dr$ ./b.out

Please enter the value of the radius: 23

Radius is :                               23.00
Circumference is :                         144.51
Area is :                                  1661.91
kimjihwan@cspro:~/test_dr$
```

함수 선언(Function Declaration)

- 함수의 헤더부분을 프로그램 앞부분에 적어주는 것
- 컴파일러에게 함수의 이름과, 함수가 갖는 parameter의 개수와 데이터형, 그리고 함수에 의해 리턴되는 결과값의 데이터형을 알려줌.
- return-type function-name(formal-parameter list)
- formal 과 actual 전달인자는 타입, 순서, 갯수는 일치해야 하지만 이름은 일치하지 않아도 됨.

함수 선언

```
int sum(int a, int b);
```



sum 함수의 존재를 미리 컴파일러에게 알려준다.

함수 호출

```
int main(void)
{
    sum(10, 20);
}
```

sum 함수가 뒤에 정의되어 있는 것을 알지 못한다.

컴파일 에러

함수 정의

sum 함수 정의

sum 함수 수행 & 결과값 반환

함수 정의(Function Definition)

- 함수는 일반적으로 다음과 같이 정의

```
return_자료형(type) 함수이름 (Formal-parameter list)
{
    변수선언 및 초기화
    구문들
    ....
    type에 맞는 리턴
}
```

```
float calculate_Circumference(const float PI, float radius)
{
    float result;
    result = 2 * PI * radius;
    return result;
}
```

- Return-type**
 - 함수의 수행의 끝나고 caller에게로 결과값을 리턴할 때 그 결과값의 데이터형 (type) 지정 (예. int, float, char, etc.)
 - 만일 caller에게 리턴하는 결과값이 없는 경우 return-type으로 키워드 void 사용

함수 정의(Function Definition)

- Formal-parameter list

- caller 가 함수를 호출할 때 그 함수(callee)에서 사용할 데이터 함께 전달 가능
- Parameter list는 comma-separated list로 각 argument는 순서대로 해당 parameter로 전달, 즉 대입됨.
- 예:

```
#include <stdio.h>

float calculate_Circumference(const float, float);

int main(void)
{
    float circ;
    float radius;
    const float PI = 3.1416;

    printf("\nPlease enter the value of the radius: ");
    scanf("%f", &radius);

    circ = calculate_Circumference(PI, radius);

    printf("\nRadius is : %10.2f", radius);
    printf("\nCircumference is : %10.2f", circ);
    printf("\n");

    return 0;
}

float calculate_Circumference(const float PI, float radius)
{
    float result;
    result = 2 * PI * radius;
    return result;
}
```

caller

callee

Actual-parameter list

Formal-parameter list

Return-type

함수 정의(Function Definition)

- **Declarations and statements : function body(block)**
 - ◆ 함수의 내용에 해당
 - ◆ 블록 안에서 변수의 선언이 가능 (can be nested)
 - ◆ 함수는 다른 함수 안에서 정의될 수 없음
- **Returning control**
 - ◆ 리턴 값이 없을 경우
 - `return;`
 - ◆ 리턴 값이 있을 경우
 - `return expression;`

Function return type should be explicitly defined

```
int first (...)  
{  
    ...  
    return (x + 2);  
} // first
```

```
void second (...)  
{  
    ...  
    return;  
} // second
```

A return statement should be used even if nothing is returned

함수 정의(Function Definition)

- 함수호출에서 형식 매개변수 (formal parameters) 와 실 매개변수 (actual parameters) 의 **개수와 타입 및 순서**는 반드시 일치해야 함
- 실제 매개변수와 형식 매개변수의 이름은 같을 필요가 없으며, 순서에 따라 서로 연결
- 예:

```
circ = calculate_Circumference(PI, radius);
```

```
float calculate_Circumference(const float PI, float radius)
```

```
float calculate_Circumference(const float john, float sam)
```

```
float calculate_Circumference(const float PI, float radius, float gang)
{
    float result;
    result = 2 * PI * radius;
    return result;
}
```



실습 5

실습 6

제어흐름과 자료흐름(control flow & data flow)

■ 제어흐름 (control flow)

- 프로그램 수행 순서를 제어흐름(control flow)라고 함
- 한 함수 내에서 제어흐름은 제어구조에 의해 결정됨
- 함수 사이의 제어흐름은 함수호출에 의해 결정됨

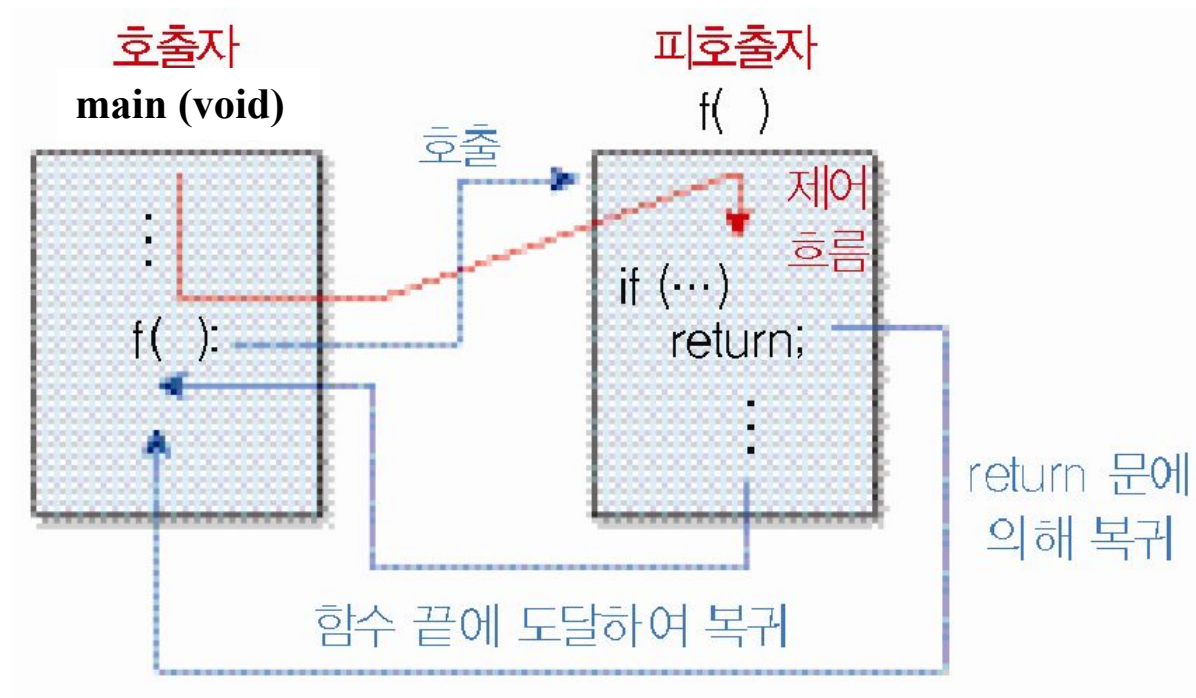
■ 자료흐름 (data flow)

- 데이터가 전달되는 순서를 자료흐름(data flow)라고 함
- 한 함수 내에서 자료흐름은 대입문에 의해 결정됨
- 함수 사이의 자료흐름은 인수전달(parameter passing)과 반환값 전달에 의해 결정됨

함수호출 메커니즘
함수 호출, 복귀 시에는
제어흐름과 자료흐름이 함께
변경되므로 주의해야 한다.

제어흐름(control flow)

- main 함수에서 f를 호출했다고 하면, f는 두 가지 방식으로 복귀할 수 있다.



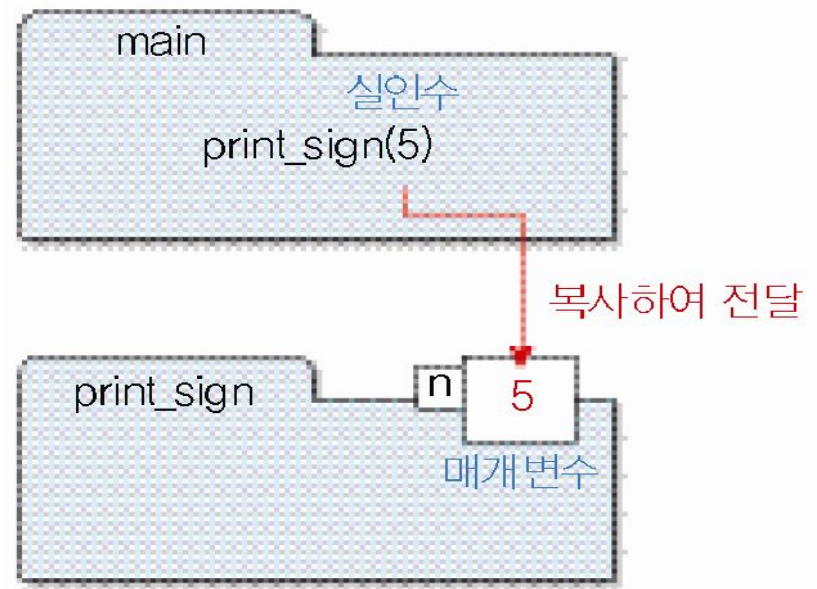
자료흐름(data flow) – 인수 전달

- 함수 인수(argument)는 값을 구하여 매개변수(parameter)에 "복사"하여 전달한다.
- 예:

```
void print_sign(int n)
{
    printf("정수 %d는 ", n);
    if (n > 0)
        printf("양수입니다.\n");
    else if (n < 0)
        printf("음수입니다.\n");
    else
        printf("양수도 음수도 아닙니다.\n");
}
```

main 함수에서
print_sign(5);를 호출했을 경우

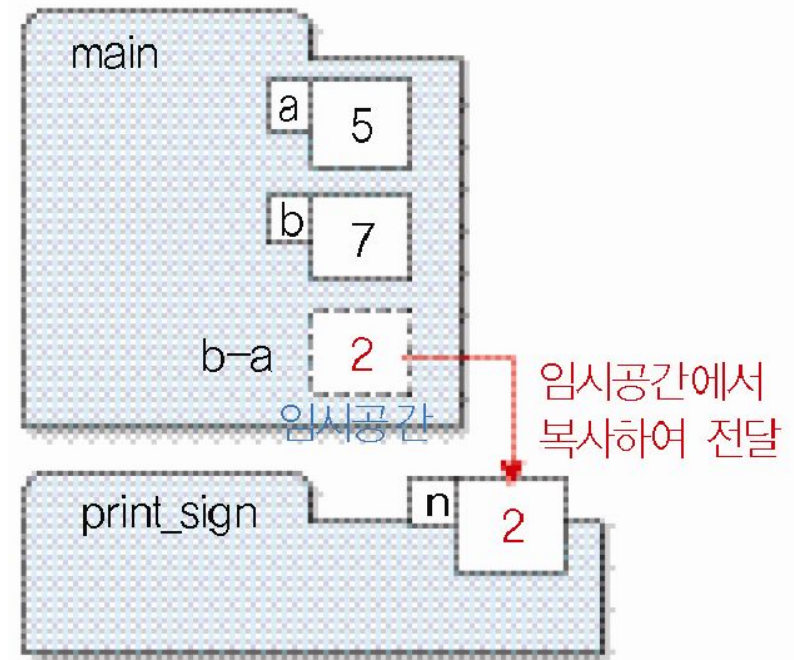
print_sign 호출자



자료흐름(data flow) – 임시변수

- 인수가 수식인 경우에는 호출자의 임시공간(임시변수)를 활용하여 수식 값을 구한 후 전달한다.
- 예:
 - 다음과 같이 호출한 경우 수식 $b - a$ 값은 main의 임시공간에서 계산된다.

```
int main (void)
{
    int a=5;
    int b=7;
    print_sign(b-1);
    return 0;
}
```



자료흐름(data flow) – 리턴값

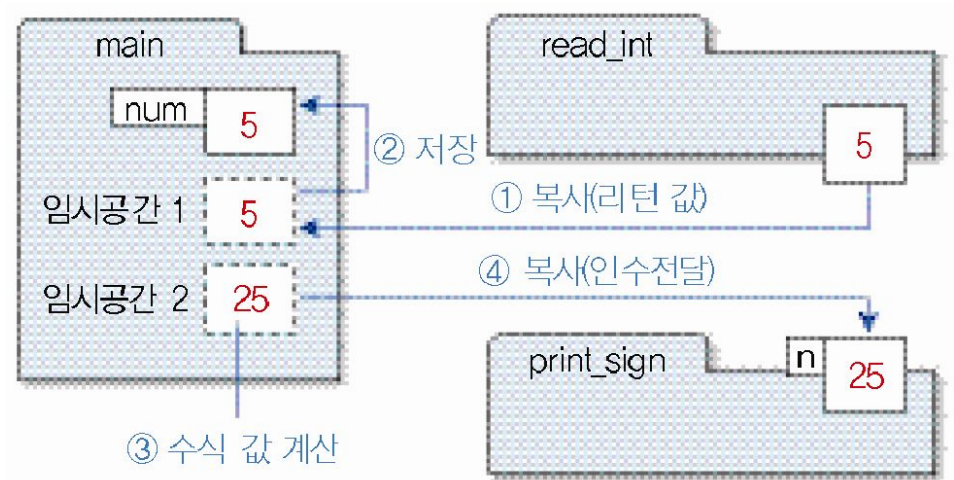
■ 값 반환 함수 호출

- 값 반환 함수 호출은 수식으로 간주하기 때문에 리턴값도 역시 호출자의 임시변수에 저장한다.
- 값 반환 함수는 사용자 정의 '연산자'라는 사실을 기억하자.

■ 예:

- 다음과 같이 호출한 경우 `read_int()` 값과 `num*num` 값은 임시공간에 저장된다.

```
int main (void)
{
    int num;
    num = read_int();
    print_sign(num*num);
    return 0;
}
```



함수 호출 메커니즘

- 함수 f가 함수 g를 호출했을 때, 함수호출 메커니즘
 - (1) f의 본체 내에서 g의 실 인수 값을 계산한다.
 - (2) g에 대한 데이터 영역이 생성된다.
 - (3) 실 인수 값을 g의 매개변수로 복사한다. 인수가 여러 개일 때, 복사 순서는 컴파일러마다 다르다.
 - (4) 제어흐름이 f에서 g로 전달된다. f는 수행을 멈춘다.
 - (5) g의 본체 끝에 도달하거나 수행 중 return 문에 도달할 때까지 g의 본체 내 문장을 수행한다.
 - (6) 리턴 수식이 있는 return 문에 도달한 경우 리턴 값을 구한다.
 - (7) 리턴 값을 f 영역 내의 임시변수에 복사한다.
 - (8) 제어흐름이 g에서 f로 전달된다. g 영역은 소멸된다.
 - (9) f의 본체가 이전에 멈추었던 위치 바로 다음부터 계속 수행된다.

예제 프로그램 2 - 나눗셈

예제 소스

```
#include <stdio.h>

int Calc_devide(int, int);
int Calc_remainder(int, int);

int main (void)
{
    int intNum1;
    int intNum2;
    int intCalc;

    printf("Enter two integral number: ");
    scanf("%d %d", &intNum1, &intNum2);

    intCalc = Calc_devide(intNum1, intNum2);
    printf("%d / %d is %d", intNum1, intNum2, intCalc);

    intCalc = Calc_remainder(intNum1, intNum2);
    printf(" with a remainder of: %d\n", intCalc);

    return 0;
}

int Calc_devide(int in1, int in2)
{
    int result;
    result = in1 / in2;
    return result;
}

int Calc_remainder(int in1, int in2)
{
    int result;
    result = in1 % in2;
    return result;
}
```

```
kimjihwan@csp:~/test_dr$ gcc -o calc.o calc.c
kimjihwan@csp:~/test_dr$ ./calc.o
Enter two integral number: 13 2
13 / 2 is 6 with a remainder of: 1
kimjihwan@csp:~/test_dr$
```

결과

실습 7

실습 8